Universidad Nacional de Córdoba

FCEFyN

Arquitectura de Computadoras

Unidad Aritmético-Lógica

Trabajo Práctico Nº 1

Diseño de una `ALU` parametrizable en `SystemVerilog`, integración y verificación sobre una placa `Basys 3`.

Institución

Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

Repositorio

`github.com/FrancoPerotti/computer-architecture-tps`

Docente

Martín Pereyra

Carrera

Ingeniería en Computación

Integrantes

Arnaudo, Federico Andrés

Perotti, Franco José

2026

Índice

1 Introducción **4**
2 Objetivos y alcance **5**
2.1 Objetivo general 5
2.2 Objetivos específicos 5
2.3 Alcance 5
3 Arquitectura general **6**
3.1 Separación de responsabilidades 6
3.2 Flujo de operación 6
3.3 Decisiones de diseño 8
3.4 Organización del proyecto 9
4 Núcleo de la ALU **11**
4.1 Contrato del núcleo 11
4.2 Operaciones implementadas 11
4.3 Interpretación de los indicadores 12
4.4 Descripción combinacional 12
5 Interfaz con la Basys 3 . . . . . **15**
5.1 El problema de las entradas manuales 15
5.2 Registros de entrada y reset 16
5.3 Distribución de controles e indicadores 17
5.4 Conexiones físicas y restricciones 18
6 Verificación funcional **20**
6.1 Criterio de validación 20
6.2 Verificación de la ALU 21
6.2.1 Criterio y estímulos 21
6.2.2 Resultado observado 21
6.2.3 Interpretación 22
6.3 Verificación del sincronizador 22
6.3.1 Criterio y estímulos 22
6.3.2 Resultado observado 22
6.3.3 Interpretación 23
6.4 Verificación de la integración 23
6.4.1 Criterio y estímulos 23
6.4.2 Resultado observado 23
6.4.3 Interpretación 24
7 Síntesis e implementación **25**
7.1 Flujo sobre el dispositivo objetivo 25
7.2 Del esquema RTL a los recursos físicos 26
7.3 Utilización de recursos 29
8 Timing, limitaciones y validación física **32**
8.1 Qué se comprueba 32
8.2 Resultado posterior al ruteo 32

8.3 Validación sobre la placa 33
9 Conclusiones **38**
Referencias **39**

Sección 1

Introducción

Una unidad aritmético-lógica reúne las operaciones que un procesador realiza sobre sus datos. Su diseño requiere establecer qué operaciones se admiten, cómo se seleccionan y de qué manera se interpretan los resultados cuando el ancho de los operandos es limitado. A partir de este problema, en el presente trabajo se desarrolló una `ALU` parametrizable en `SystemVerilog` para la `FPGA Artix-7` de una placa `Basys 3`.

El núcleo implementado realiza ocho operaciones sobre datos de ocho bits: suma, resta, `AND`, `OR`, `XOR`, `NOR`, desplazamiento lógico a derecha y desplazamiento aritmético a derecha. Además del resultado, genera indicadores de cero, acarreo y desbordamiento con signo. El circuito se integró con una interfaz que permite cargar los operandos y el código de operación desde los switches de la placa y visualizar la respuesta mediante sus LEDs.

El desarrollo recorrió las etapas de descripción, simulación, síntesis e implementación. Para aislar responsabilidades se construyeron tres módulos: el núcleo combinacional, un sincronizador para las entradas asíncronas de los pulsadores y un módulo superior que almacena los valores ingresados. Cada nivel dispone de un `testbench` propio, con el que se verificaron tanto las operaciones como la secuencia de interacción.

Con el comportamiento funcional ya comprobado, comenzó el flujo de `Vivado`. Primero se elaboraron los esquemáticos y se sintetizó el diseño para el dispositivo `xc7a35tcpg236-1` de la placa. Después se implementaron sus conexiones y se analizaron la utilización de recursos y los márgenes temporales. El informe presenta la relación entre el código, las estructuras inferidas por la herramienta y la evidencia obtenida en las pruebas.

Sección 2

Objetivos y alcance

2.1 Objetivo general

El objetivo del trabajo práctico es diseñar, describir y verificar una unidad aritmético-lógica capaz de operar sobre datos binarios de ancho parametrizable, para luego integrarla con los recursos físicos disponibles en una placa `Basys 3`.

2.2 Objetivos específicos

El desarrollo contempla los siguientes objetivos:

- Describir en `SystemVerilog` una lógica combinacional que implemente ocho operaciones aritméticas, lógicas y de desplazamiento.
- Generar indicadores de resultado cero, acarreo sin signo y desbordamiento en complemento a dos.
- Separar el núcleo de cálculo de la interfaz secuencial utilizada para ingresar datos desde la placa.
- Sincronizar los pulsadores externos antes de utilizarlos en el dominio del reloj de $100\,\mathrm{MHz}$.
- Verificar por separado la `ALU`, el sincronizador y el módulo superior, incluyendo casos límite y estímulos pseudoaleatorios reproducibles.
- Sintetizar e implementar el circuito en la `Artix-7` de la placa y contrastar los recursos obtenidos con la arquitectura prevista.
- Analizar el cumplimiento temporal de las rutas sujetas a las restricciones declaradas.

2.3 Alcance

El trabajo documenta la descripción RTL, los archivos de restricciones y 866 comprobaciones automáticas. También registra la síntesis, la ubicación y el ruteo del circuito. El recorrido se completa con la generación del `bitstream` y la validación física de trece casos sobre una placa `Basys 3`.

Los resultados de simulación respaldan los casos ensayados, aunque no enumeran todas las combinaciones posibles. De manera análoga, el análisis temporal solo alcanza las rutas gobernadas por el reloj. La prueba física comprueba el procedimiento de carga y la observación de resultados en los LEDs, pero tampoco reemplaza un ensayo exhaustivo de las entradas ni una caracterización eléctrica de los pulsadores.

Sección 3

Arquitectura general

3.1 Separación de responsabilidades

La primera decisión del desarrollo consiste en evitar que la operación aritmética quede mezclada con los detalles propios de la placa. La `ALU` solo necesita recibir dos operandos y un código de operación; no debería conocer de dónde provienen esos datos ni cómo se presentan al usuario. Los switches, pulsadores, registros y LEDs pertenecen a otra responsabilidad.

Con ese criterio, el cálculo combinacional queda a cargo de `alu`. El módulo `button_input` incorpora las entradas manuales al dominio del reloj, mientras que `alu_top` conserva los valores ingresados y conecta el conjunto con la `Basys 3`. Esta separación permite verificar cada problema de manera aislada antes de probar la integración.

La figura 3.1 resume el recorrido completo. Los ocho switches forman un bus compartido: el usuario prepara un valor y el pulsador correspondiente decide en qué registro debe guardarse. Una vez almacenados A, B y el opcode, la ALU actualiza de manera combinacional el resultado y los indicadores.

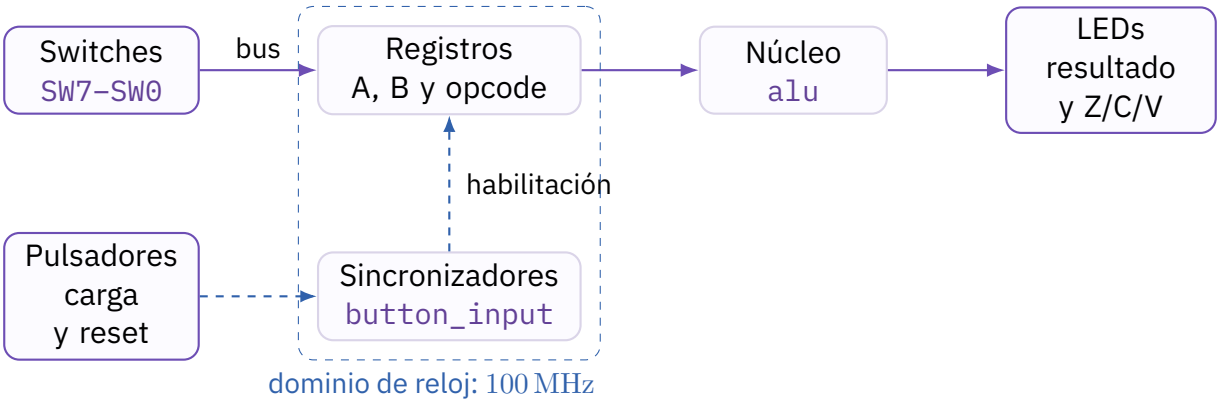


Figura 3.1: Arquitectura conceptual y flujo de datos del sistema.

3.2 Flujo de operación

El ingreso se realiza en tres pasos. Primero se configura A en los switches y se pulsa el botón izquierdo. Luego se repite el procedimiento para B con el botón derecho. Por último se colocan los seis bits del opcode y se utiliza el botón superior. Los pulsadores no escriben directamente en el núcleo aritmético. Antes de habilitar los registros que conservan cada valor, atraviesan los sincronizadores.

Esta diferencia es importante. Los operandos pueden cargarse en momentos distintos porque los registros mantienen su contenido entre pulsaciones. Cuando el último valor cambia, la lógica combinacional recalcula inmediatamente la salida. El usuario no necesita una orden adicional de ejecución: cargar el opcode completa la entrada y los LEDs reflejan el resultado.

Etapa	Módulo responsable	Descripción
Ingreso manual	Recursos de la Basys 3	Los switches presentan un valor y los pulsadores eligen su destino.
Sincronización	`button_input`	Cada pulsador atraviesa dos flip-flops antes de utilizarse como control.
Almacenamiento	`alu_top`	Tres registros conservan A, B y opcode; el reset tiene prioridad.
Cálculo	`alu`	La lógica combinacional selecciona una de ocho operaciones y genera Z/C/V.
Presentación	`alu_top`	Los ocho bits de resultado y los tres indicadores se conectan a los LEDs.

Tabla 3.1: Flujo funcional desde los controles hasta la salida.

3.3 Decisiones de diseño

Las decisiones de la tabla 3.2 establecen tanto el comportamiento del circuito como las condiciones bajo las cuales debe utilizarse.

Decisión	Motivo	Consecuencia
Separar `alu` de `alu_top`	Aislar el cálculo de la interfaz física.	El núcleo puede verificarse sin reloj ni controles de placa.
Compartir los ocho switches	Reutilizar una interfaz limitada para tres entradas.	A, B y opcode deben cargarse de forma secuencial.
Sincronizar cada pulsador con dos flip-flops	Reducir el riesgo de propagar metastabilidad.	La habilitación aparece varios ciclos después de la acción manual.
Usar cargas activas por nivel	Mantener una lógica de control simple.	Una pulsación sostenida recarga el mismo valor; no se genera un pulso único.
No implementar antirrebote	Las recargas son inocuas si el bus permanece estable.	Los switches no deben modificarse hasta que el botón se haya liberado.
Definir cero para opcodes inválidos	Cubrir todas las combinaciones y evitar memoria implícita.	El indicador Z se activa ante un código no reconocido.
Reservar carry para ADD	Mantener una semántica inequívoca del indicador.	SUB no informa préstamo; C permanece en cero.
Dar prioridad al reset	Asegurar un estado conocido incluso si hay cargas activas.	A, B y opcode se borran mientras el reset sincronizado esté activo.
Exceptuar switches y LEDs del timing	Modelar que no poseen una relación temporal externa definida.	El reporte temporal no demuestra la estabilidad manual del bus ni el retardo hacia las salidas.

Tabla 3.2: Decisiones de diseño, fundamentos e implicancias.

3.4 Organización del proyecto

La organización de las fuentes reproduce la arquitectura. Cada módulo de diseño posee un banco de pruebas dedicado y las restricciones físicas permanecen separadas del resto del diseño.

Archivo	Función
`alu.sv`	Operaciones de la ALU e indicadores.
`button_input.sv`	Sincronización de un pulsador.
`alu_top.sv`	Registros e integración con la placa.
`alu_tb.sv`	Verificación del núcleo combinacional.
`button_input_tb.sv`	Verificación del sincronizador.
`alu_top_tb.sv`	Verificación de cargas, reset e integración.
`basys3.xdc`	Pines y restricciones temporales.
`scripts/create_sync_project.tcl`	Creación reproducible del proyecto de Vivado.

Tabla 3.3: Archivos principales del trabajo práctico.

La jerarquía mostrada por `Vivado` en la figura 3.2 confirma esa división. Bajo `alu_top` aparecen cuatro instancias de `button_input`, una por pulsador, y la instancia `alu_core`. Los testbenches se encuentran en *Simulation Sources* y no forman parte del circuito sintetizado.

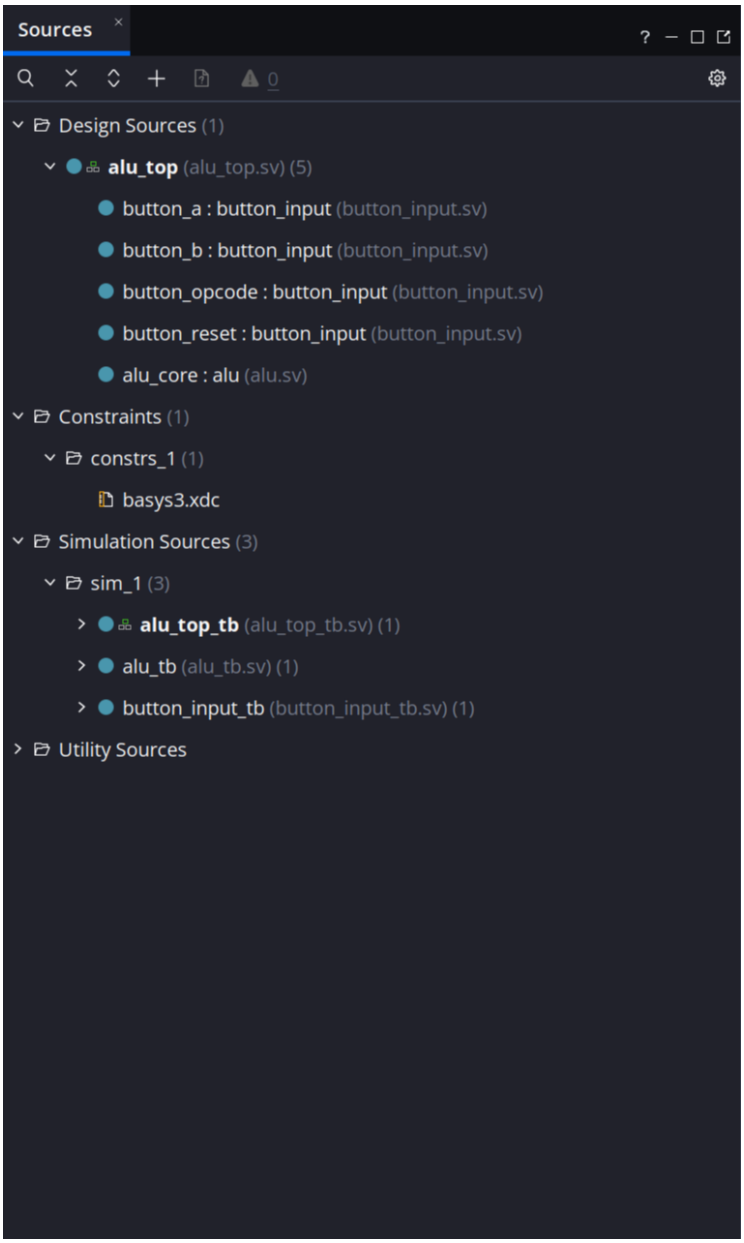


Figura 3.2: Jerarquía de módulos, restricciones y fuentes de simulación.

Sección 4

Núcleo de la ALU

4.1 Contrato del núcleo

Una vez separada la interfaz de la placa, el contrato de la ALU resulta pequeño: recibe dos operandos y un código de operación, y entrega un resultado acompañado por tres indicadores. Los operandos y la salida comparten el ancho `NB_DATA`; el control utiliza `NB_OPCODE` bits. En la implementación utilizada para la `Basys 3` esos parámetros toman los valores 8 y 6, respectivamente.

Esta interfaz permite analizar el núcleo como una función combinacional. No existen registros internos ni señales de habilitación: cualquier cambio en A, B u opcode produce un nuevo resultado después del retardo de propagación de la lógica.

4.2 Operaciones implementadas

Los códigos elegidos y su comportamiento se resumen en la tabla 4.1. En las operaciones booleanas se aplica la función correspondiente a cada par de bits. En los desplazamientos, B representa la cantidad de posiciones que debe desplazarse A.

Operación	Opcode	Expresión	Descripción
ADD	`100000`	`A + B`	Suma
SUB	`100010`	`A - B`	Resta
AND	`100100`	`A & B`	AND bit a bit
OR	`100101`	`A | B`	OR bit a bit
XOR	`100110`	`A ^ B`	XOR bit a bit
SRA	`000011`	`$signed(A) >> B`	Desplazamiento aritmético a derecha
SRL	`000010`	`A >> B`	Desplazamiento lógico a derecha
NOR	`100111`	`~(A | B)`	NOR bit a bit

Tabla 4.1: Operaciones implementadas por la ALU.

La diferencia entre los dos desplazamientos se vuelve visible cuando el bit más significativo de A vale uno. Si A es `80` y B es `01`, `SRL` introduce un cero por la izquierda y produce `40`. Por el contrario, `SRA` interpreta A con signo y replica ese uno, por lo que obtiene `C0`.

Los códigos que no aparecen en la tabla producen cero. No se trata de una operación adicional, sino de una decisión defensiva: todas las combinaciones de la entrada quedan definidas y el circuito nunca necesita conservar el resultado anterior.

4.3 Interpretación de los indicadores

Con ocho bits, un mismo patrón puede representar un valor entre 0 y 255 sin signo o uno entre -128 y 127 en complemento a dos. Carry y overflow describen desbordamientos diferentes; por eso no deben interpretarse como indicadores equivalentes. La tabla 4.2 resume la condición de activación de cada salida.

Salida	Indicador	Condición
`o_zero`	Cero, Z	Todos los bits del resultado son cero.
`o_carry`	Acarreo, C	ADD genera un bit adicional fuera del ancho de la salida.
`o_overflow`	Desbordamiento, V	ADD o SUB excede el rango representable con signo.

Tabla 4.2: Indicadores generados por el núcleo.

El caso `FF + 01` muestra el carry. Interpretados sin signo, los operandos valen 255 y 1; el resultado completo es 256 y necesita nueve bits. La salida conserva `00`, Z se activa porque esos ocho bits son cero y C conserva el bit adicional. En complemento a dos, en cambio, la misma entrada representa $-1 + 1 = 0$, por lo que V permanece apagado.

La suma `7F + 01` presenta la situación inversa. El valor 128 cabe en ocho bits sin signo y no genera carry, pero supera el máximo positivo en complemento a dos. La salida `80` se interpretaría como -128; V advierte que ese patrón ya no representa la suma con signo esperada.

La misma regla alcanza la resta. En `80 - 01`, el resultado matemático con signo es -129, que queda fuera del rango. Los ocho bits conservados forman `7F` y V se activa. C permanece en cero porque en este trabajo no se utiliza como indicador de préstamo.

A	B	Operación	Resultado	Z	C	V
`05`	`03`	ADD	`08`	0	0	0
`FF`	`01`	ADD	`00`	1	1	0
`7F`	`01`	ADD	`80`	0	0	1
`80`	`80`	ADD	`00`	1	1	1
`80`	`01`	SUB	`7F`	0	0	1
`7F`	`FF`	SUB	`80`	0	0	1
`05`	`05`	SUB	`00`	1	0	0

Tabla 4.3: Casos representativos para distinguir Z, C y V.

4.4 Descripción combinacional

El núcleo se implementa con un bloque `always_comb` y una sentencia `case`. Carry y overflow reciben un valor predeterminado antes de seleccionar la operación; ADD y SUB los reemplazan cuando

corresponde. El caso `default` asigna el resultado nulo. Esta estructura garantiza que todas las salidas tengan un valor en cada evaluación y evita inferir latches.

```
always_comb begin
o_carry = 1'b0;
o_overflow = 1'b0;

case (i_data_opcode)
OP_ADD: begin
{o_carry, o_result} =
{1'b0, i_data_a} + {1'b0, i_data_b};
o_overflow =
(i_data_a[NB_DATA-1] == i_data_b[NB_DATA-1]) &&
(o_result[NB_DATA-1] != i_data_a[NB_DATA-1]);
end

OP_SUB: begin
o_result = i_data_a - i_data_b;
o_overflow =
(i_data_a[NB_DATA-1] != i_data_b[NB_DATA-1]) &&
(o_result[NB_DATA-1] != i_data_a[NB_DATA-1]);
end

// Operaciones logicas y desplazamientos.
default: o_result = {NB_DATA{1'b0}};
endcase
end
```

En ADD, la suma se amplía un bit para conservar el acarreo sin signo. El overflow se obtiene comparando los signos de los operandos y de la salida. SUB utiliza una condición análoga, pero parte de operandos con signos distintos. Las operaciones restantes se expresan directamente mediante los operadores de `SystemVerilog`; `$signed` permite que SRA replique el signo.

El indicador Z no depende de la operación elegida, sino únicamente del patrón final. Por eso se calcula con una asignación común fuera del bloque:

```
assign o_zero = (o_result == {NB_DATA{1'b0}});
```

La figura 4.1 permite cerrar el recorrido entre descripción y circuito. A la izquierda aparecen las funciones aritméticas, lógicas y de desplazamiento; sus salidas llegan a multiplexores controlados por el opcode. A la derecha se reconoce la lógica que selecciona los indicadores. El esquema no agrega una arquitectura nueva: es la interpretación que `Vivado` realiza del contrato y de las expresiones anteriores.

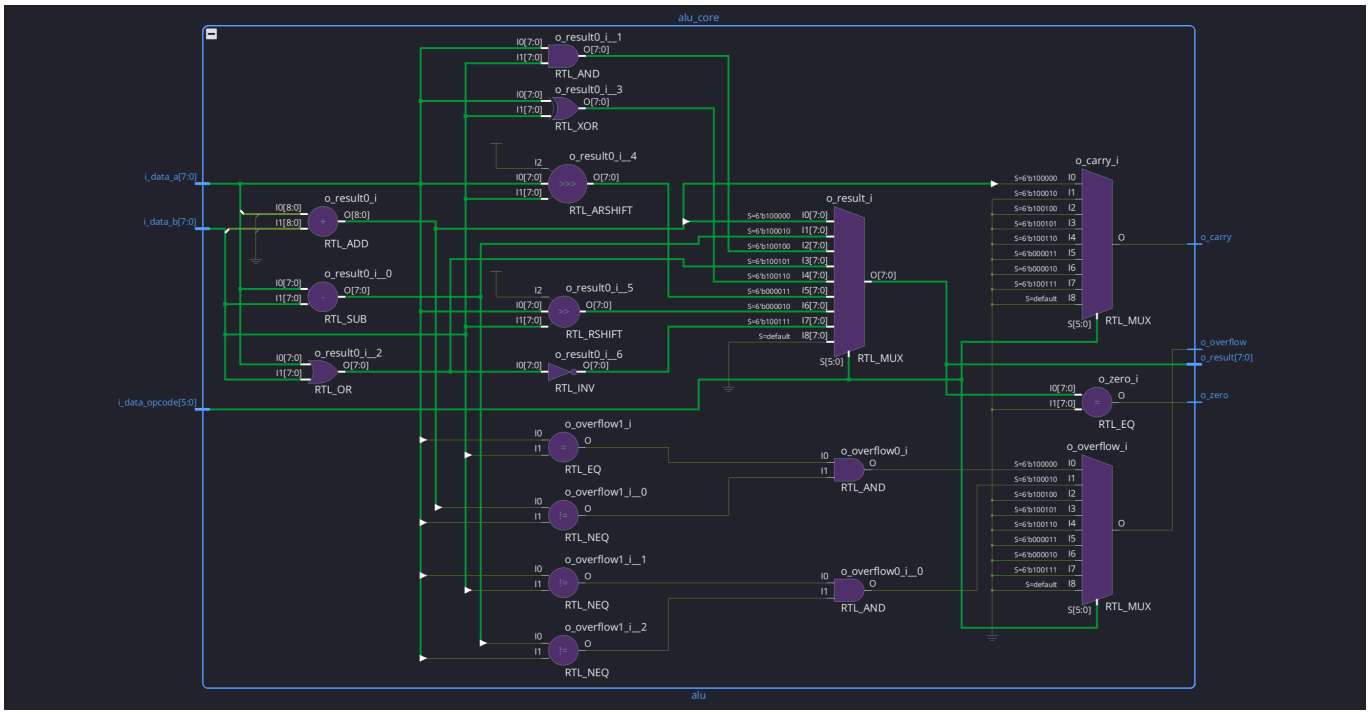


Figura 4.1: Operaciones, selección del resultado e indicadores en la vista RTL.

Sección 5

Interfaz con la Basys 3

5.1 El problema de las entradas manuales

La ALU puede describirse sin reloj, pero utilizarla desde la placa introduce un problema adicional. Los pulsadores cambian cuando el usuario los acciona y, por lo tanto, sus transiciones no guardan relación con el reloj de $100\,\mathrm{MHz}$. Si una de ellas ocurre cerca de un flanco, el primer registro que la recibe puede entrar temporalmente en metastabilidad.

El módulo `button_input` reduce el riesgo de que ese estado se propague mediante dos flip-flops en serie. La primera etapa captura el pin y dispone de casi un ciclo para estabilizarse; la segunda recién utiliza esa muestra en el flanco siguiente.

```
(* ASYNC_REG = "TRUE" *) reg button_meta = 1'b0;
(* ASYNC_REG = "TRUE" *) reg button_sync = 1'b0;

always @(posedge clk) begin
button_meta <= button;
button_sync <= button_meta;
end

assign button_pressed = button_sync;
```

Las asignaciones no bloqueantes son esenciales: ambos flip-flops calculan su próximo estado a partir de los valores anteriores al flanco. La segunda etapa no recibe inmediatamente la muestra nueva de la primera. El atributo `ASYNC_REG` informa a `Vivado` que ambos registros forman un sincronizador y permite que la herramienta preserve esa relación durante la implementación.

La figura 5.1 muestra la conexión que resulta de la descripción. Esta estructura sincroniza un nivel, pero no elimina rebotes ni genera un pulso de un solo ciclo. Esa diferencia determina el modo de uso: los switches deben permanecer estables mientras el botón está presionado y hasta que su liberación haya atravesado las dos etapas.

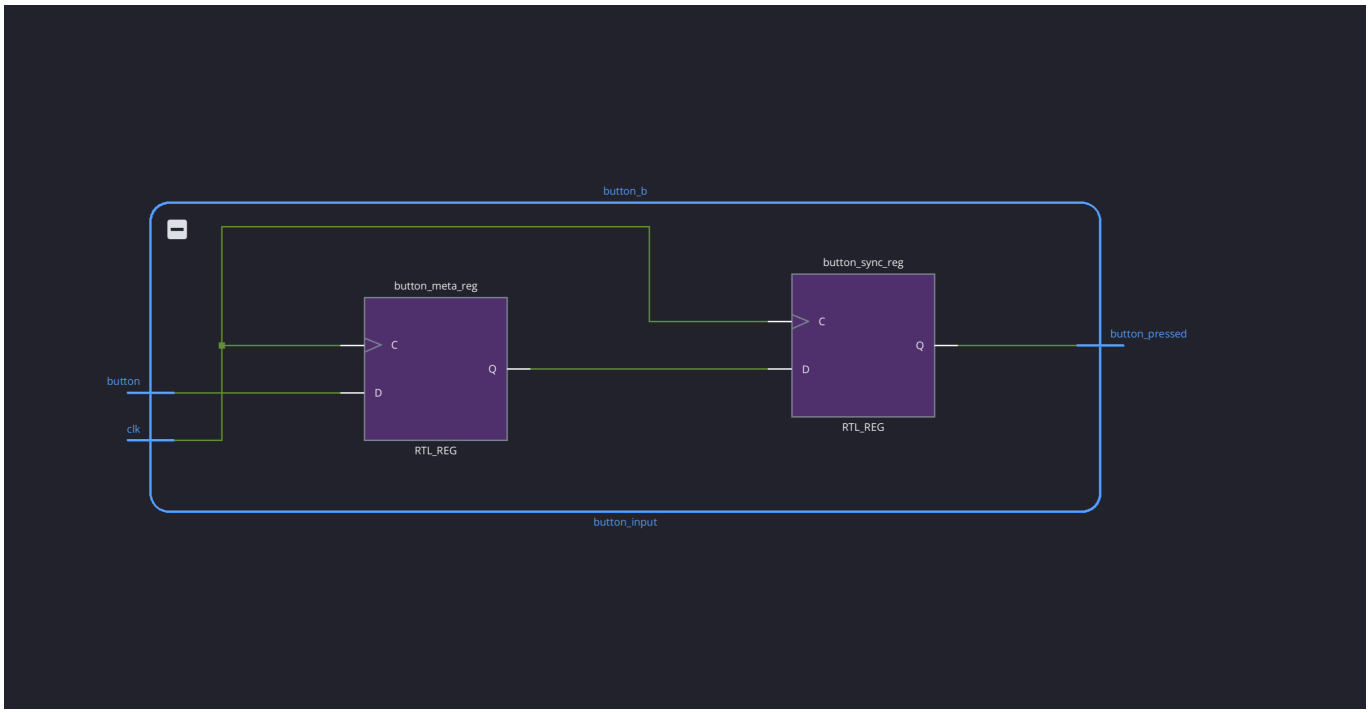


Figura 5.1: Cadena de dos registros utilizada para sincronizar un pulsador.

5.2 Registros de entrada y reset

Una vez dentro del dominio del reloj, las señales de control sirven como habilitaciones de `alu_top`. A y B ocupan ocho bits y el opcode utiliza los seis switches inferiores. Cada registro conserva su contenido mientras su habilitación permanece inactiva.

El reset tiene prioridad sobre las tres cargas. La señal `startup_reset`, que comienza en uno, complementa al botón central sincronizado y garantiza que los registros se borren en el primer flanco posterior a la configuración.

```
reg startup_reset = 1'b1;

always @(posedge clk) begin
startup_reset <= 1'b0;

if (startup_reset || reset_pressed) begin
data_a <= 8'b0;
data_b <= 8'b0;
opcode <= 6'b0;
end else begin
if (load_a)      data_a <= switches;
if (load_b)      data_b <= switches;
if (load_opcode) opcode <= switches[5:0];
end
end
```

Durante el primer flanco, el `if` todavía observa el valor inicial de `startup_reset`; su actualización a cero se aplica al finalizar el paso. En ciclos posteriores decide el pulsador central. Como las cargas se evalúan con tres condiciones independientes, es posible cargar A y B simultáneamente con el mismo valor.

La vista RTL del módulo superior reúne estas piezas en la figura 5.2. Allí, los sincronizadores llegan a las habilitaciones de los registros y el bus de switches alimenta sus entradas. Las salidas registradas se dirigen al núcleo combinacional, del que parten los once bits conectados a los LEDs.

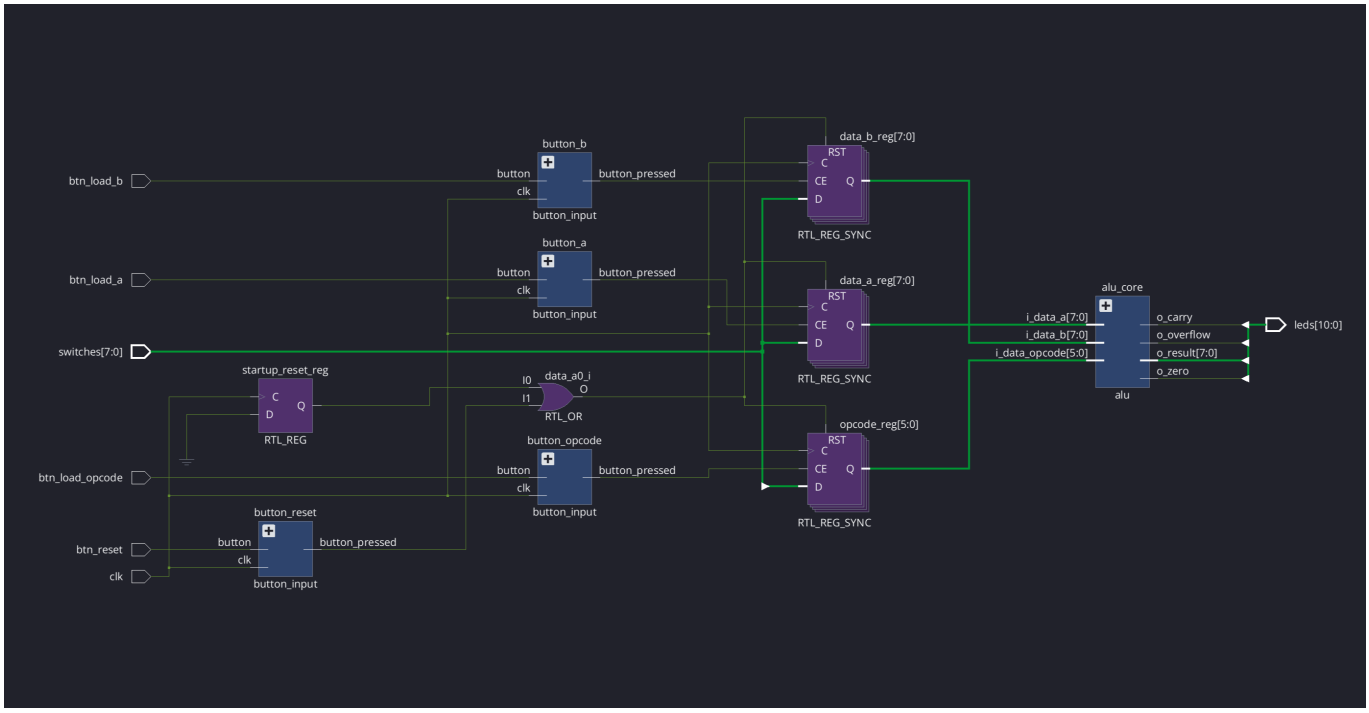


Figura 5.2: Integración de sincronizadores, registros y núcleo de la ALU.

5.3 Distribución de controles e indicadores

La ubicación de los botones acompaña la secuencia de carga. BTNL guarda A, BTNR guarda B y BTNU guarda el opcode. BTNC restablece el estado inicial. El botón inferior queda libre.

Recurso	Puerto	Función
SW7–SW0	`switches[7:0]`	Dato compartido para las cargas.
BTNL	`btn_load_a`	Cargar A.
BTNR	`btn_load_b`	Cargar B.
BTNU	`btn_load_opcode`	Cargar opcode desde SW5–SW0.
BTNC	`btn_reset`	Borrar A, B y opcode.
LED7–LED0	`leds[7:0]`	Resultado de ocho bits.
LED8	`leds[8]`	Indicador Z.
LED9	`leds[9]`	Indicador C.
LED10	`leds[10]`	Indicador V.

Tabla 5.1: Asignación funcional de los recursos de la Basys 3.

Para calcular $5 + 3$, por ejemplo, se coloca `00000101` y se utiliza BTNL para almacenarlo. A continuación, se utiliza BTNR para cargar el valor `00000011`. Por último, se configura `100000` en SW5–SW0 y se utiliza BTNU para guardar el opcode. Los registros contienen entonces las tres entradas de ADD y LED3 se enciende para representar `00001000`. Al pulsar BTNC, los registros vuelven a cero y LED8 indica el resultado nulo.

5.4 Conexiones físicas y restricciones

El archivo `basys3.xdc` relaciona la interfaz funcional con los pines de la placa. El reloj ingresa por W5 y se declara con un período de $10\,\mathrm{ns}$. Los pulsadores de A, B, opcode y reset se conectan a W19, T17, T18 y U18. El resultado ocupa LED0 a LED7, mientras que Z, C y V se asignan a LED8, LED9 y LED10.

La figura 5.3 reúne los 24 bits de entrada o salida y permite revisar tanto el pin físico como el estándar `LVCMOS33`.

Tcl Console | Messages | Log | Reports | Design Runs | I/O Ports

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
All ports (24)													
switches (8)	IN			✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[7]	IN		W13	✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[6]	IN		W14	✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[5]	IN		V15	✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[4]	IN		W15	✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[3]	IN		W17	✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[2]	IN		W16	✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[1]	IN		V16	✓	14	LVCMOS33*	3.300				NONE	NONE	
switches[0]	IN		V17	✓	14	LVCMOS33*	3.300				NONE	NONE	
leds (11)	OUT			✓	(Multiple)	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[10]	OUT		W3	✓	34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[9]	OUT		V3	✓	34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[8]	OUT		V13	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[7]	OUT		V14	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[6]	OUT		U14	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[5]	OUT		U15	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[4]	OUT		W18	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[3]	OUT		V19	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[2]	OUT		U19	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[1]	OUT		E19	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
leds[0]	OUT		U16	✓	14	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
Scalar ports (5)													
btn_load_a	IN		W19	✓	14	LVCMOS33*	3.300				NONE	NONE	
btn_load_b	IN		T17	✓	14	LVCMOS33*	3.300				NONE	NONE	
btn_load_op	IN		T18	✓	14	LVCMOS33*	3.300				NONE	NONE	
btn_reset	IN		U18	✓	14	LVCMOS33*	3.300				NONE	NONE	
clk	IN		W5	✓	34	LVCMOS33*	3.300				NONE	NONE	

Figura 5.3: Asignación de puertos, pines y estándar eléctrico.

Las restricciones temporales también documentan supuestos del entorno. Los pulsadores se consideran asíncronos hasta la primera etapa del sincronizador. Los switches se exceptúan porque el procedimiento exige configurarlos antes de la carga, y los LEDs no poseen un reloj externo que defina un instante de captura. Las rutas internas entre registros continúan sujetas al reloj de la placa.

Esta distinción impide atribuir al reporte temporal más alcance del que realmente tiene: un timing interno correcto no comprueba que el usuario haya mantenido estable el bus ni mide el retardo combinacional hasta uno de los LEDs.

Sección 6

Verificación funcional

6.1 Criterio de validación

La verificación se organiza con la misma separación que el diseño. Primero se comprueba el cálculo combinacional; luego, el ingreso de una señal asíncrona al dominio del reloj; y finalmente, el recorrido completo desde los controles hasta los LEDs. De esta manera, una falla puede asociarse con una responsabilidad concreta antes de analizar el conjunto.

El análisis estático complementó esas pruebas. En la ejecución documentada, el panel *Linter* de `Vivado` no informó violaciones ni reglas exceptuadas. Este resultado respalda la calidad de la descripción, aunque no demuestra por sí mismo que las operaciones sean correctas. La figura 6.1 documenta esta ejecución.

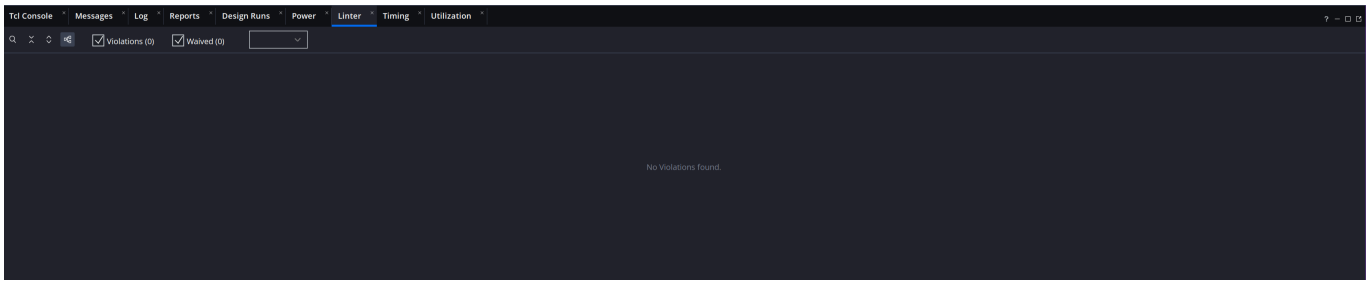


Figura 6.1: Análisis estático de Vivado sin violaciones reportadas.

La tabla 6.1 relaciona los requisitos principales con el módulo que los implementa y la evidencia utilizada.

Requisito	Módulo	Verificación	Evidencia
Ejecutar ocho operaciones	`alu`	`alu_tb`	Casos dirigidos y 100 pares pseudoaleatorios por operación.
Generar Z, C y V	`alu`	`alu_tb`	Fronteras con y sin signo y comparación con un modelo ampliado.
Sincronizar pulsadores	`button_input`	`button_input_tb`	Activación y liberación observadas después de dos flancos.
Cargar y conservar entradas	`alu_top`	`alu_top_tb`	Cargas individuales, simultáneas y botón sostenido.
Priorizar el reset	`alu_top`	`alu_top_tb`	Reset aplicado junto con las tres cargas.
Depender del reloj	`alu_top`	`alu_top_tb`	Registros conservados con el reloj detenido.
Cumplir $100\,\mathrm{MHz}$	Diseño implementado	Timing post-route	WNS, WHS y WPWS positivos en las rutas restringidas.

Tabla 6.1: Trazabilidad entre requisitos y evidencia de verificación.

6.2 Verificación de la ALU

6.2.1 Criterio y estímulos

Verificar la ALU exige algo más que comprobar que aparezca un valor en la salida. Es necesario confirmar, por un lado, que cada opcode seleccione la operación prevista y, por otro, que los indicadores Z, C y V conserven su significado en los casos límite.

El banco `alu_tb.sv` organiza cada ensayo mediante la tarea `check_result`. La tarea presenta A, B y el opcode, deja transcurrir $10\,\mathrm{ns}$ para que se estabilice la lógica combinacional y recién entonces contrasta la respuesta completa con un modelo de referencia. La comparación utiliza `!==`; por eso un bit desconocido no puede pasar inadvertido como si fuera un resultado válido.

El modelo de referencia está planteado, además, para no repetir literalmente las expresiones utilizadas por el circuito. Si ambos lados resolvieran cada condición de la misma manera, un error podría quedar reproducido tanto en el diseño como en la prueba. Para calcular el overflow, por ejemplo, los operandos se extienden con signo a nueve bits. La suma ampliada puede compararse entonces con los límites -128 y 127 sin perder el desbordamiento que se quiere detectar. El circuito obtiene el mismo indicador mediante los bits de signo, de modo que ambas implementaciones llegan al resultado por caminos diferentes.

Con el carry se aplica el mismo criterio de independencia. El modelo determina que existe acarreo cuando A supera el valor que resulta de restar B al máximo sin signo. Por su parte, la ALU conserva el noveno bit de la suma. Así, el testbench no solo compara resultados, sino también dos formas equivalentes de interpretar sus condiciones de borde.

```
signed_a = {test_a[NB_DATA-1], test_a};
signed_b = {test_b[NB_DATA-1], test_b};

expected = test_a + test_b;
expected_carry = test_a > (ALL_ONES - test_b);
signed_result = signed_a + signed_b;
expected_overflow =
(signed_result < SIGNED_MIN) ||
(signed_result > SIGNED_MAX);
```

La secuencia incluye 14 casos dirigidos: resultado cero, carry, fronteras positivas y negativas, desplazamientos, NOR y opcode inválido. A continuación se generan cien pares de operandos para cada una de las ocho operaciones. La semilla fija `32'h1a2b3c4d` permite repetir exactamente los 800 estímulos pseudoaleatorios.

6.2.2 Resultado observado

La figura 6.2 muestra el comienzo de la ejecución. `FF + 01` entrega `00`, con Z y C activos; luego, el mismo patrón `80` desplazado una posición produce `C0` con SRA, mientras que con SRL da `40`. La diferencia entre ambas salidas hace visibles tres propiedades diferentes sin depender todavía de los estímulos aleatorios.

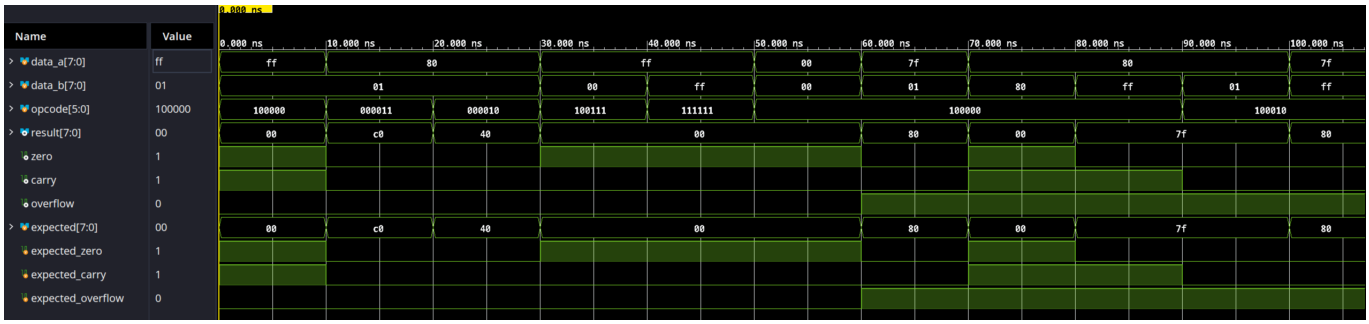


Figura 6.2: Primeros casos dirigidos de la ALU y sus indicadores.

La comparación automática continuó durante toda la simulación. La consola confirmó que las 814 comprobaciones terminaron sin errores a los $8140\,\mathrm{ns}$. El cierre de la ejecución puede verse en la figura 6.3.

```
source alu_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'create_wave_config' in the TCL console."
#   }
# }
# run all
PASS: 814 tests completed without errors
$finish called at time : 8140 ns : File "/home/franco/college/computer_architecture/labs/tps/tp1/alu_tb.sv" Line 186
INFO: [USF-XSim-96] XSim completed. Design snapshot 'alu_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for all
```

Figura 6.3: Finalización de las 814 comprobaciones de la ALU.

6.2.3 Interpretación

El resultado confirma que las ocho ramas, el caso inválido y las combinaciones dirigidas de Z, C y V coinciden con el modelo utilizado. Las pruebas pseudoaleatorias amplían la variedad de operandos, mientras que los casos de frontera aseguran que no se dependa del azar para alcanzar los escenarios más importantes. No se trata de una enumeración exhaustiva de las 2^{22} combinaciones posibles de A, B y opcode, pero sí de una cobertura dirigida y reproducible de las propiedades definidas.

6.3 Verificación del sincronizador

6.3.1 Criterio y estímulos

La prueba de `button_input` busca demostrar que la salida no sigue al pin de manera inmediata, sino después de atravesar las dos etapas. El testbench genera un reloj de $10\,\mathrm{ns}$, modifica el botón en flancos descendentes y observa la salida después de cada flanco ascendente. También aplica un nivel sostenido, la liberación y una pulsación de un solo ciclo.

6.3.2 Resultado observado

En la figura 6.4, la entrada se activa a los $10\,\mathrm{ns}$. La primera etapa la captura a los $15\,\mathrm{ns}$ y la salida cambia a los $25\,\mathrm{ns}$. Al liberar el botón a los $80\,\mathrm{ns}$, la salida vuelve a cero a los $95\,\mathrm{ns}$. Las 14 comprobaciones finalizaron sin errores a los $126\,\mathrm{ns}$.

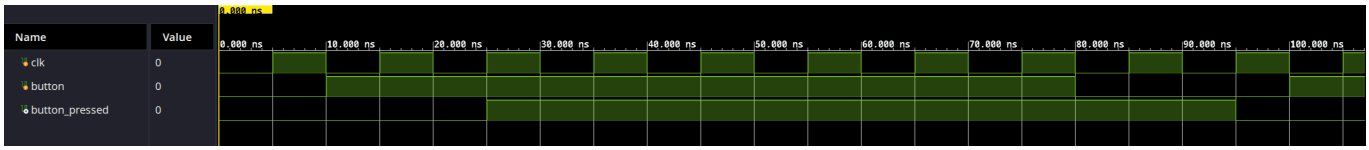


Figura 6.4: Propagación del nivel por las dos etapas del sincronizador.

```
source button_input_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'create_wave_config' in the TCL console."
#   }
# }
# run all
PASS: 14 button checks completed without errors
$finish called at time : 126 ns : File "/home/franco/college/computer_architecture/labs/tps/tp1/button_input_tb.sv" Line 93
INFO: [USF-XSim-96] XSim completed. Design snapshot 'button_input_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for all
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:05 . Memory (MB): peak = 12100.012 ; gain = 78.074 ; free physical = 5007 ; free virtual = 21699
```

Figura 6.5: Finalización de las 14 comprobaciones del sincronizador.

6.3.3 Interpretación

La secuencia observada coincide con dos registros conectados en serie y con las asignaciones no bloqueantes utilizadas. La simulación comprueba el retardo lógico y la conservación de niveles; no reproduce la metastabilidad eléctrica que motiva la estructura ni permite medir su probabilidad. Tampoco demuestra un filtrado de rebotes, porque el módulo no cumple esa función.

6.4 Verificación de la integración

6.4.1 Criterio y estímulos

El último banco de pruebas se ocupa de la integración. Comprueba que cada valor presentado en los switches llegue al registro correcto y que el resultado aparezca en los LEDs. En cada paso compara simultáneamente A, B, opcode y los once bits de salida. Así puede distinguir un error de cálculo de una carga equivocada.

La secuencia ensaya el estado inicial, cambios de switches sin pulsación, cargas individuales, botones sostenidos, rebotes simulados, carga simultánea de A y B, prioridad del reset y recuperación posterior. También detiene el reloj durante una solicitud de carga para comprobar que los registros conserven su estado hasta que vuelvan los flancos.

El testbench reserva tres ciclos para cada carga: dos para que el nivel atraviese el sincronizador y uno adicional para que el registro observe la habilitación ya actualizada. Esta espera reproduce el comportamiento de los bloques secuenciales y no es un retardo arbitrario agregado a la prueba.

6.4.2 Resultado observado

La figura 6.6 presenta la inicialización y el comienzo de la carga de A. Mientras el opcode permanece en cero, los ocho bits de resultado son nulos y `leds` muestra `100` hexadecimal, correspondiente a Z activo. El contador de errores permanece en cero.

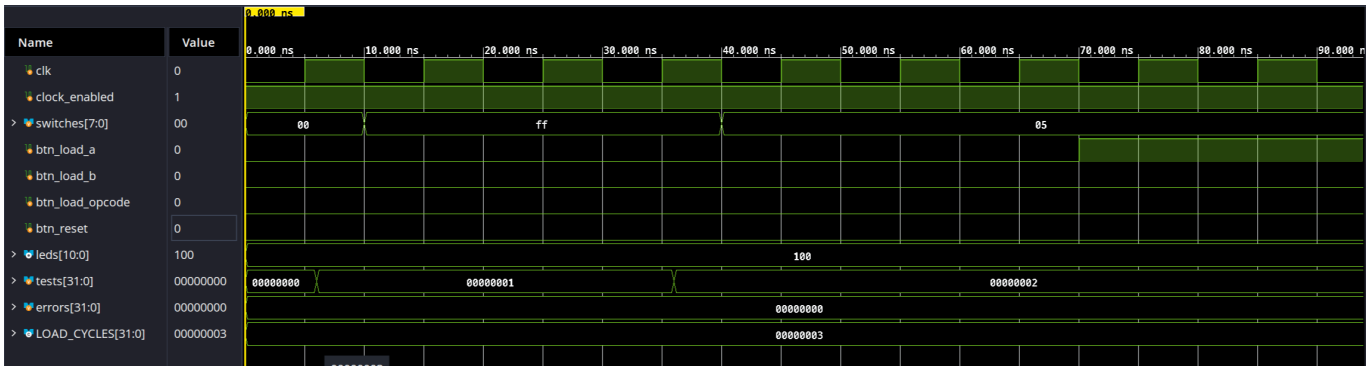


Figura 6.6: Inicialización y comienzo de una carga en el módulo superior.

Fuera de la ventana visible se aplicaron operaciones conocidas para encender y apagar cada indicador, se verificó el reset desde registros no nulos y se comprobó que el sistema pudiera volver a operar después del borrado. La ejecución concluyó sin errores después de 38 comprobaciones, a los $7051\,\mathrm{ns}$.

```
source alu_top_tb.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Waveform Configuration' or type 'create_wave_config' in the TCL console."
#   }
# }
# run all
PASS: 38 top checks completed without errors
$finish called at time : 7051 ns : File "/home/franco/college/computer_architecture/labs/tps/tp1/alu_top_tb.sv" Line 259
INFO: [USF-XSim-96] XSim completed. Design snapshot 'alu_top_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for all
launch_simulation: Time (s): cpu = 00:00:07 ; elapsed = 00:00:06 . Memory (MB): peak = 10108.633 ; gain = 106.176 ; free physical = 3899 ; free virtual = 22667
```

Figura 6.7: Finalización de las 38 comprobaciones de integración.

6.4.3 Interpretación

Estas pruebas enlazan las responsabilidades verificadas previamente: los pulsadores se propagan con el retardo previsto, los registros capturan y conservan los datos, el reset prevalece sobre las cargas y la ALU actualiza los LEDs. En conjunto, los tres bancos completaron 866 comprobaciones sin errores.

Sección 7

Síntesis e implementación

7.1 Flujo sobre el dispositivo objetivo

Con el comportamiento verificado se sintetizó el diseño para el dispositivo `xc7a35tcpg236-1`. En esta etapa `Vivado` transforma los operadores y registros de la descripción RTL en una red de recursos disponibles en la `Artix-7`. La síntesis terminó correctamente, como se muestra en la figura 7.1, y permitió avanzar a la implementación.

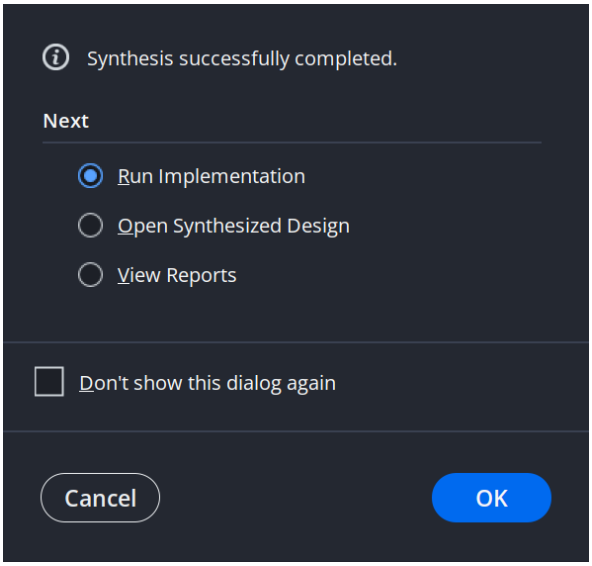


Figura 7.1: Finalización correcta de la síntesis.

Durante la implementación se determina la ubicación de los recursos y el ruteo de sus conexiones. El proceso incorpora las restricciones de pines y reloj y genera el diseño sobre el que se efectúa el análisis temporal posterior al ruteo. La figura 7.2 documenta su finalización.

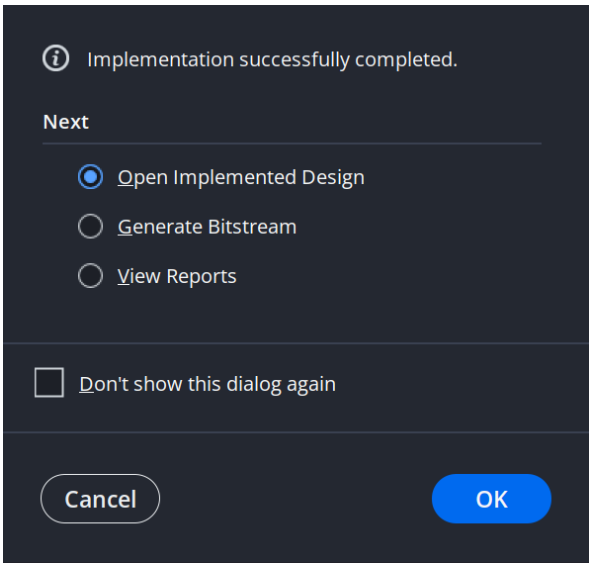


Figura 7.2: Finalización correcta de la ubicación y el ruteo.

7.2 Del esquema RTL a los recursos físicos

El esquema implementado del top, presentado en la figura 7.3, conserva el recorrido de la vista RTL, desde las entradas y los sincronizadores hasta los registros, la ALU y las salidas. La diferencia es el nivel de detalle. Los bancos aparecen descompuestos en registros individuales y se incorporan buffers de entrada, salida y distribución del reloj.

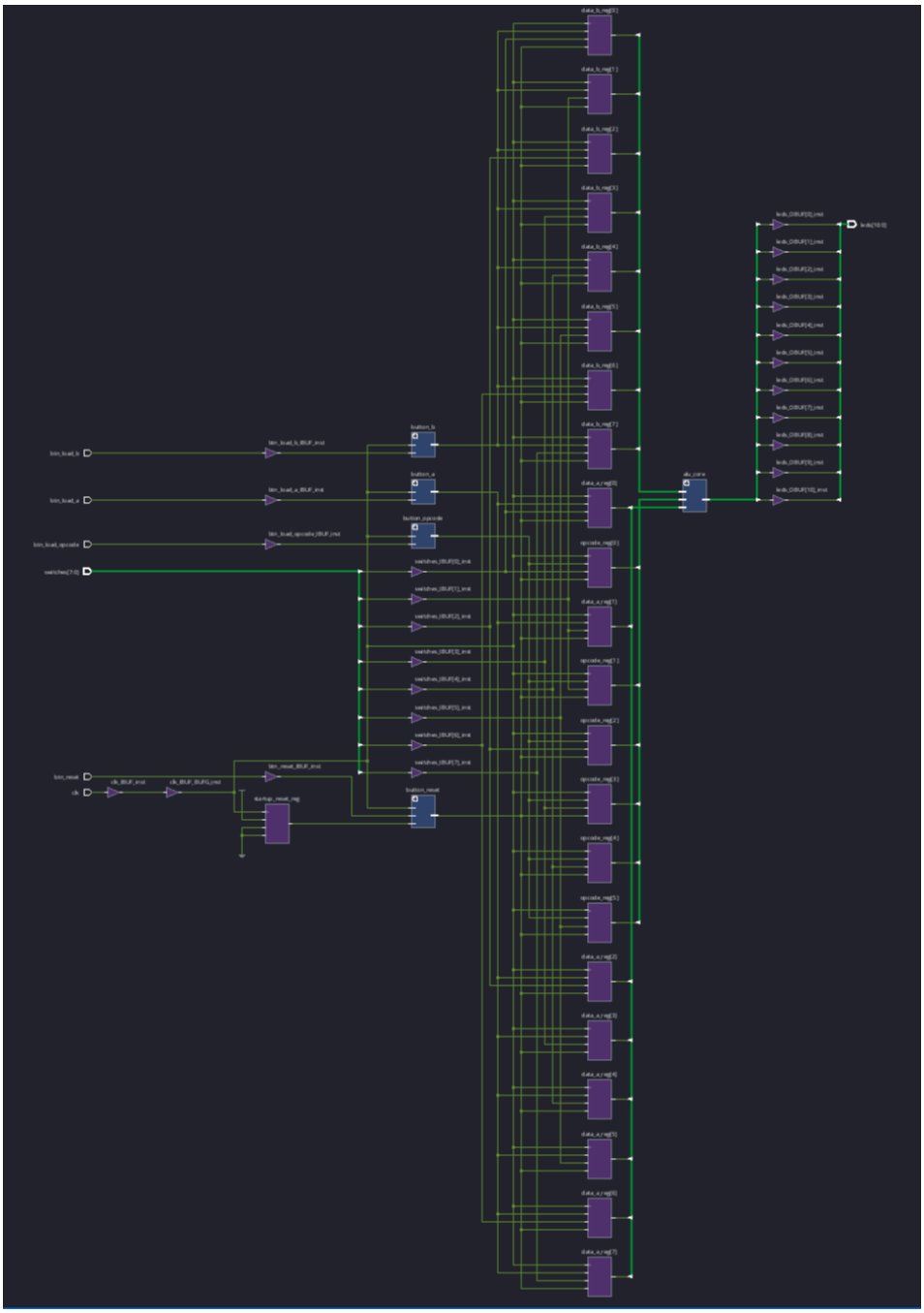


Figura 7.3: Recursos y conexiones del módulo superior implementado.

Al abrir la ALU se observa cómo la herramienta resuelve las operaciones en la tecnología del dispositivo. Las expresiones del código no se convierten necesariamente en compuertas individuales: la herramienta combina funciones dentro de las LUT y emplea cadenas dedicadas CARRY4 para la aritmética. La figura 7.4 explica, por tanto, la diferencia entre los operadores abstractos de la vista RTL y la red implementada.

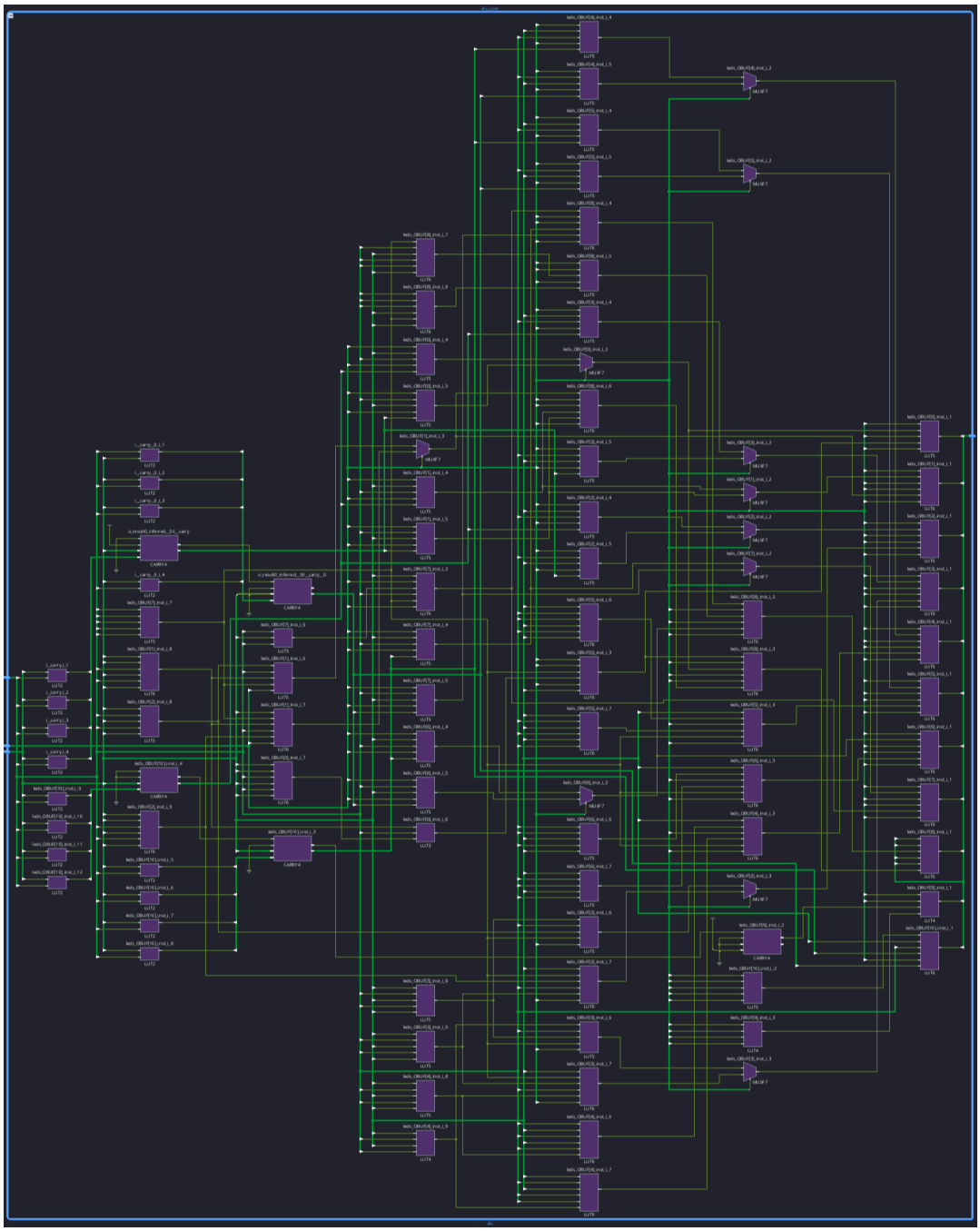


Figura 7.4: Implementación de la ALU mediante LUT, multiplexores y cadenas de acarreo.

En el sincronizador la correspondencia es más directa. Como muestra la figura 7.5, los dos registros de la descripción se implementan con dos flip-flops FDRE, con un reloj compartido y conexión en serie.

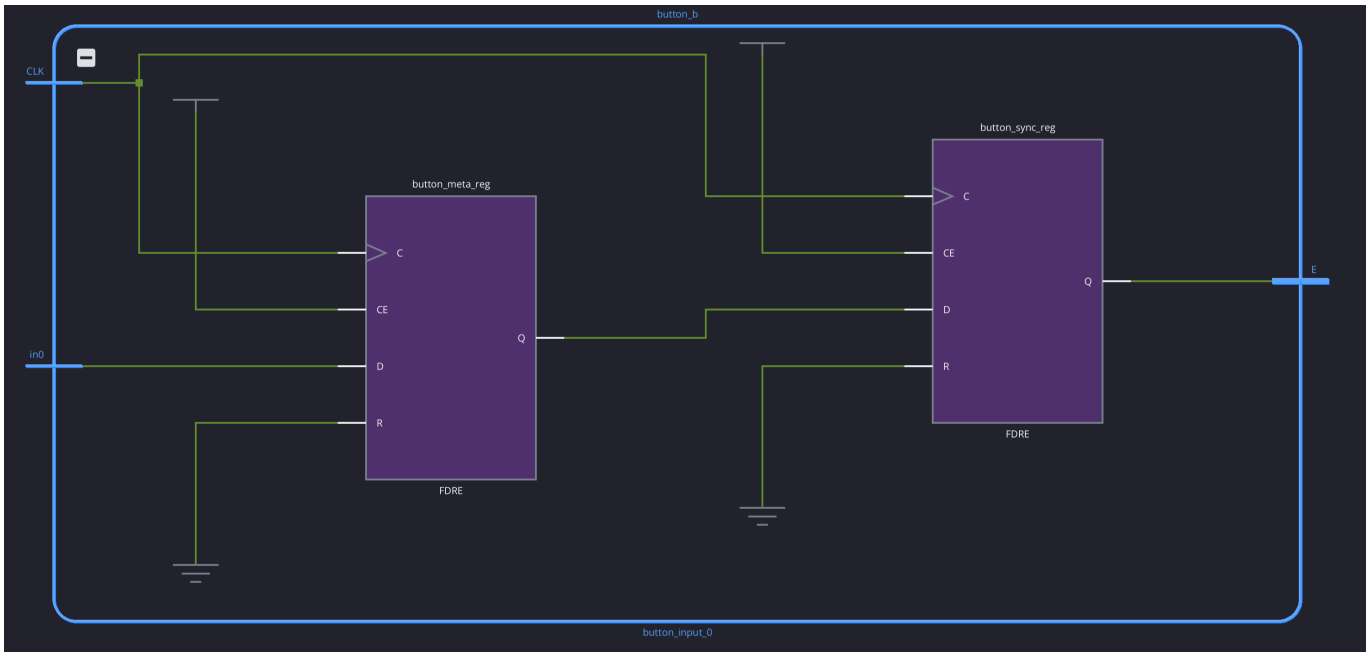


Figura 7.5: Dos flip-flops FDRE implementan las etapas del sincronizador.

7.3 Utilización de recursos

El reporte de utilización cuantifica el costo del circuito. Los resultados se resumen en la tabla 7.1.

Recurso	Utilizado	Disponible	Utilización
LUT	76	20.800	0,37 %
Flip-flops	31	41.600	0,07 %
I/O	24	106	22,64 %

Tabla 7.1: Recursos utilizados en la Artix-7.

La figura 7.6 conserva la vista completa del reporte del que se extrajeron esos valores.

Summary

Resource	Utilization	Available	Utilization %
LUT	76	20800	0.37
FF	31	41600	0.07
IO	24	106	22.64

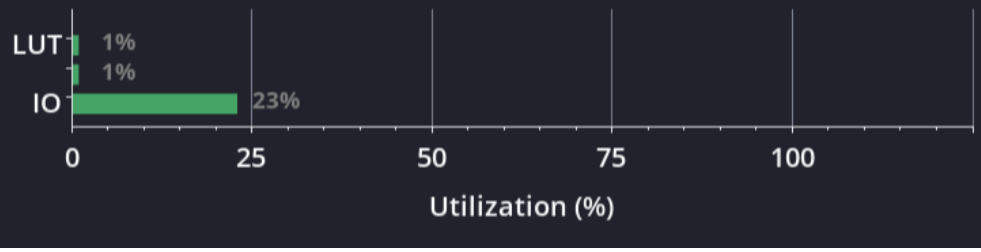


Figura 7.6: Resumen de utilización de recursos.

Los 31 flip-flops coinciden con la estructura prevista: los registros de A, B y opcode requieren 22 bits; los cuatro sincronizadores agregan ocho, y el reset de arranque utiliza uno. Los 24 I/O se corresponden con ocho switches, cuatro botones, un reloj y once LEDs. Este contraste permite validar el reporte contra las decisiones de diseño.

La vista jerárquica de la figura 7.7 indica que 75 de las 76 LUT pertenecen al núcleo y que este no contiene registros, como corresponde a su carácter combinacional. La fila del top ya incluye los recursos de sus submódulos y no debe sumarse otra vez. También aparecen once multiplexores F7 y un buffer global de reloj.

Hierarchy

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
N alu_top	76	31	11	27	76	24	1
I alu_core (alu)	75	0	11	23	75	0	0

Figura 7.7: Distribución de recursos entre el módulo superior y la ALU.

Finalmente, la vista *Device* de la figura 7.8 permite ubicar el circuito dentro de la estructura física del dispositivo programable. Los porcentajes se toman del reporte de utilización, ya que el tamaño aparente de los elementos en esta vista depende de la representación gráfica.

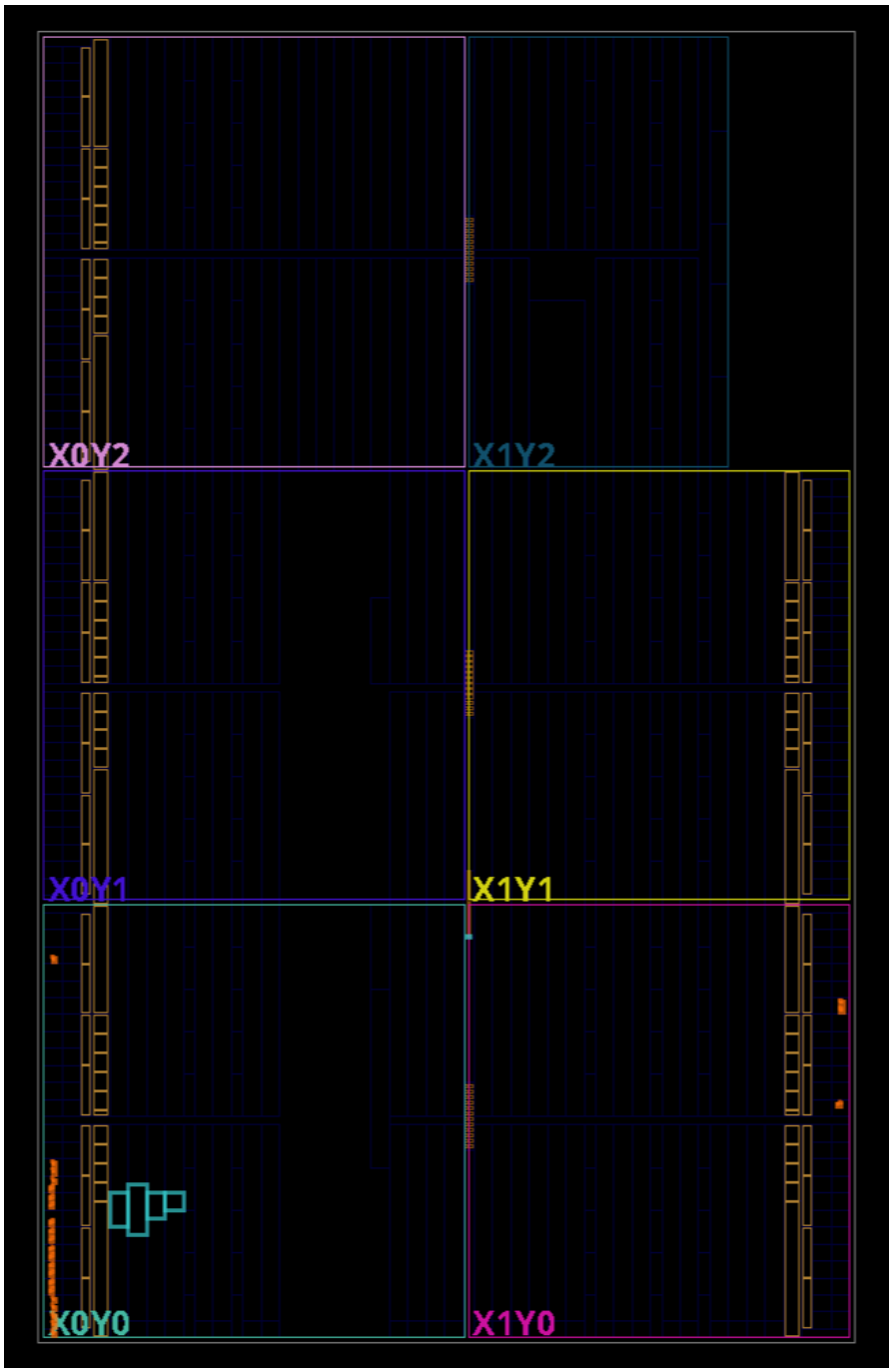


Figura 7.8: Distribución física del circuito en la vista Device.

Sección 8

Timing, limitaciones y validación física

8.1 Qué se comprueba

El análisis temporal comprueba que las rutas gobernadas por el reloj puedan operar con el período de $10\,\mathrm{ns}$ definido para la Basys 3 una vez ubicado y ruteado el circuito. A diferencia de las simulaciones anteriores, esta etapa no evalúa resultados aritméticos: analiza los retardos introducidos por los recursos y las conexiones físicas elegidas por la herramienta.

8.2 Resultado posterior al ruteo

El resumen de timing presentó márgenes positivos de setup, hold y ancho de pulso, sin endpoints fallidos. Los valores se reúnen en la tabla 8.1 y se observan en la figura 8.1.

Análisis	Peor margen	Violación total	Endpoints con falla
Setup, WNS / TNS	$7{,}224\,\mathrm{ns}$	$0{,}000\,\mathrm{ns}$	0
Hold, WHS / THS	$0{,}128\,\mathrm{ns}$	$0{,}000\,\mathrm{ns}$	0
Ancho de pulso, WPWS / TPWS	$4{,}500\,\mathrm{ns}$	$0{,}000\,\mathrm{ns}$	0

Tabla 8.1: Resumen temporal posterior al ruteo.

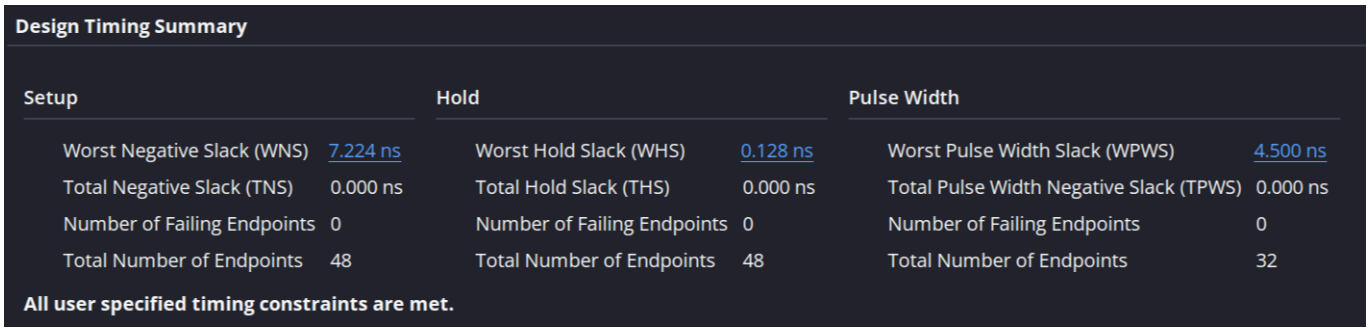


Figura 8.1: Cumplimiento de las restricciones temporales especificadas.

El WNS de $7{,}224\,\mathrm{ns}$ indica que la ruta de setup más exigente conserva ese margen antes del límite de captura. El WHS de $0{,}128\,\mathrm{ns}$ confirma que los datos también permanecen estables durante el intervalo mínimo posterior al flanco. El análisis consideró 48 endpoints para setup y hold y 32 para ancho de pulso, todos sin violaciones.

El resultado permite afirmar que las rutas internas incluidas por las restricciones cumplen el reloj de $100\,\mathrm{MHz}$.

8.3 Validación sobre la placa

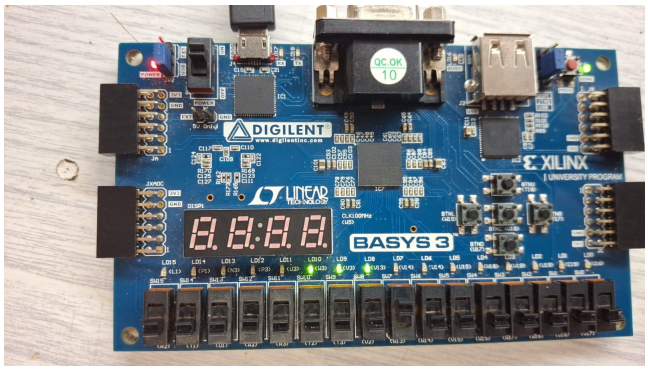
Finalmente, la validación física se realizó sobre la `Basys 3`. El bitstream se programó correctamente y el diseño funcionó en el primer intento: no fue necesario modificar el RTL, las restricciones ni la secuencia de carga. Los switches permitieron ingresar A, B y el opcode mediante sus pulsadores correspondientes, y los LEDs mostraron en todos los casos el resultado y los indicadores previstos.

Para conservar la relación entre cada fotografía y sus estímulos, los archivos se nombraron como `OPERACION_A_B.jpeg`, con los operandos expresados en hexadecimal. La tabla 8.2 reúne los trece casos observados. Las columnas Z, C y V corresponden, respectivamente, a cero, carry y overflow.

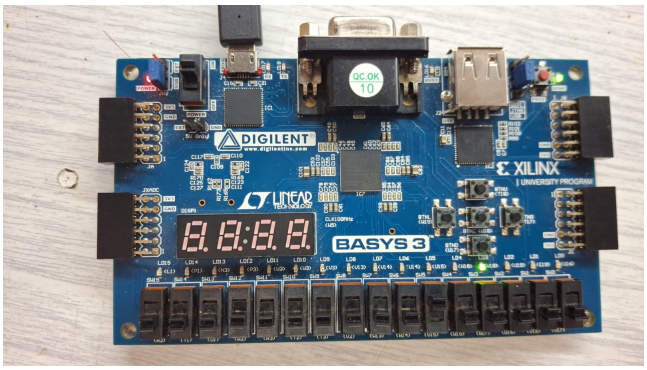
Operación	A	B	Resultado	Z	C	V
ADD	80	80	00	1	1	1
ADD	05	03	08	0	0	0
ADD	7F	01	80	0	0	1
ADD	FF	01	00	1	1	0
SUB	05	05	00	1	0	0
SUB	7F	FF	80	0	0	1
SUB	80	01	7F	0	0	1
AND	0C	0A	08	0	0	0
OR	A0	0F	AF	0	0	0
XOR	AA	FF	55	0	0	0
NOR	F0	0F	00	1	0	0
SRL	80	01	40	0	0	0
SRA	80	01	C0	0	0	0

Tabla 8.2: Resultados observados durante la validación física.

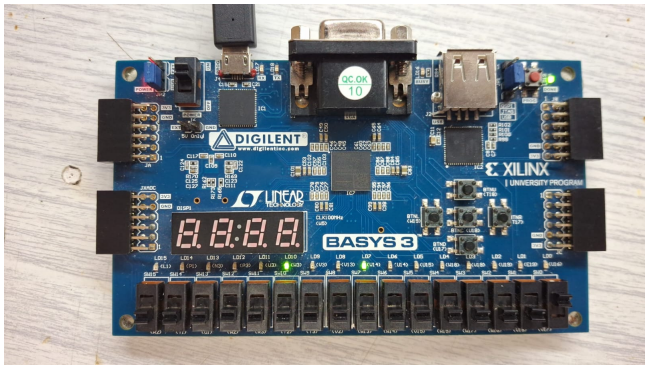
Las fotografías siguientes muestran los LEDs después de completar cada secuencia de carga. Las sumas y restas se reúnen en las figuras 8.2 y 8.3, respectivamente. Las operaciones lógicas aparecen en la figura 8.4, mientras que la figura 8.5 permite comparar ambos desplazamientos.



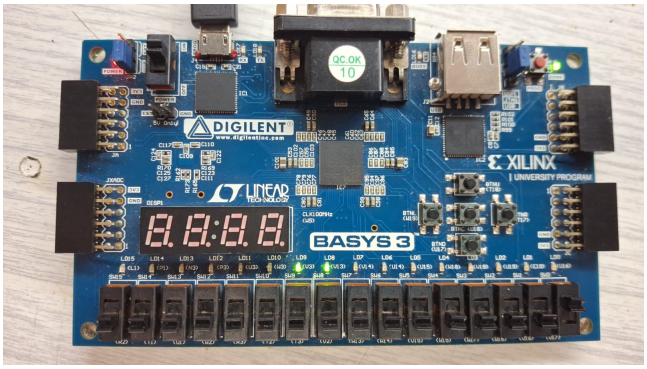
ADD 80 80



ADD 05 03

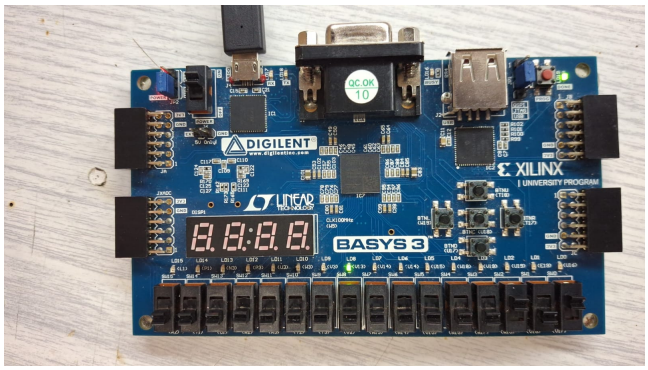


ADD 7F 01

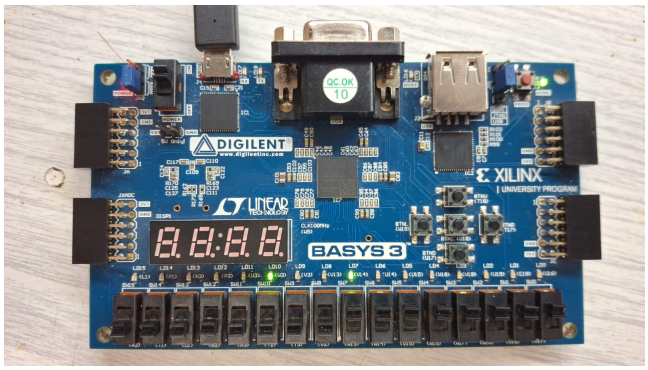


ADD FF 01

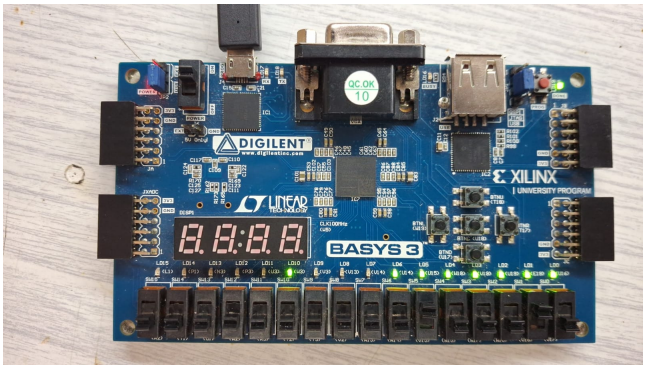
Figura 8.2: Casos físicos de suma, incluidos cero, carry y overflow.



SUB 05 05

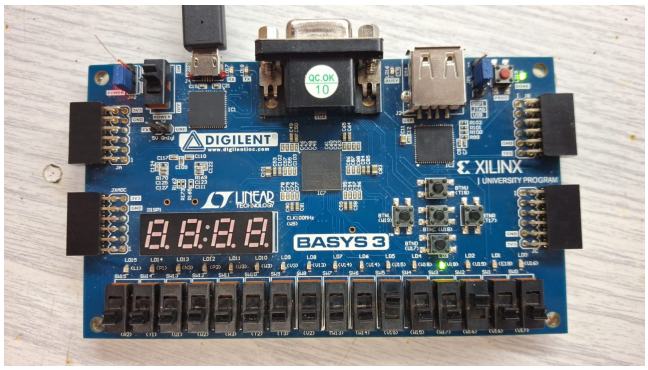


SUB 7F FF

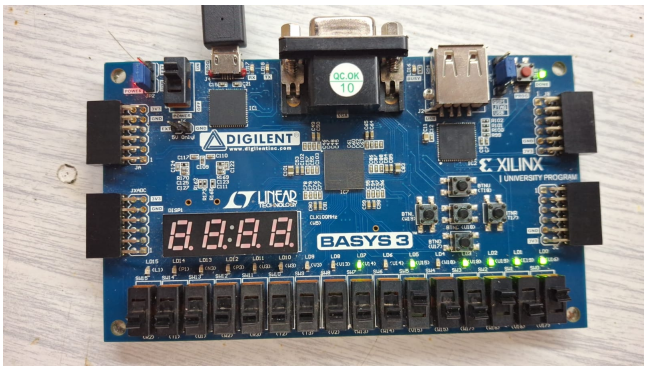


SUB 80 01

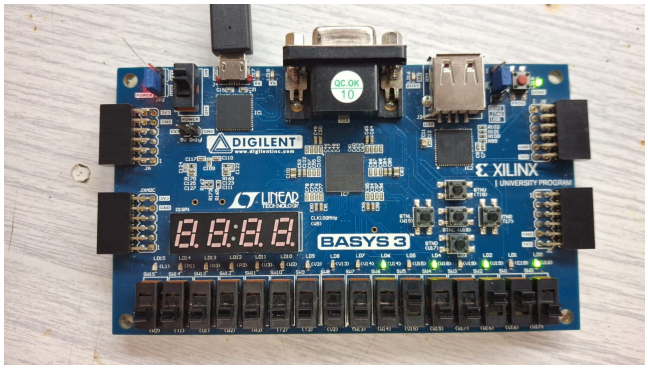
Figura 8.3: Casos físicos de resta con resultado cero y overflow.



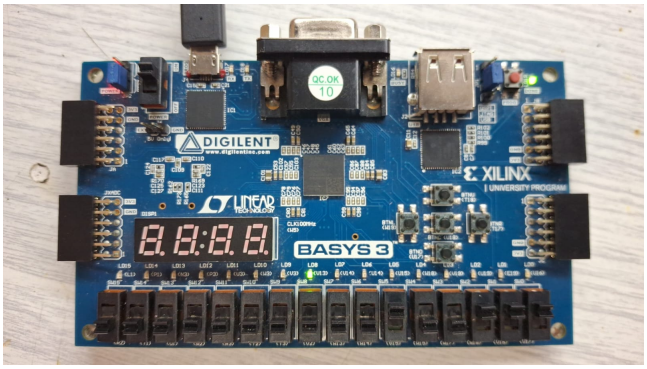
AND 0C 0A



OR A0 0F

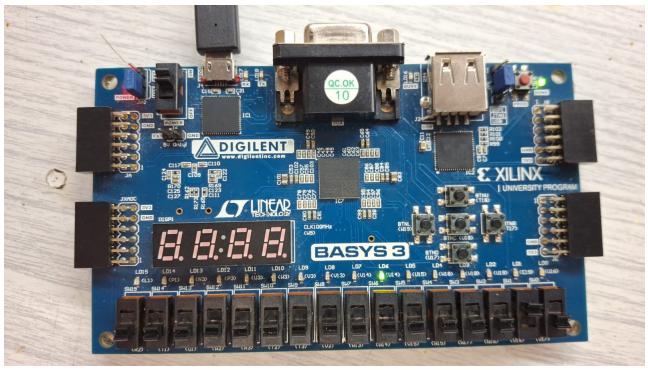


XOR AA FF

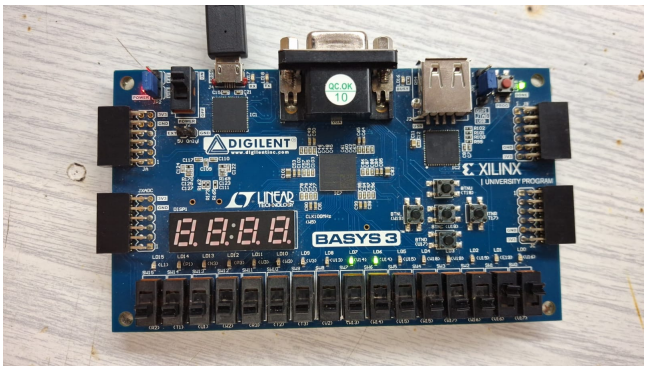


NOR F0 0F

Figura 8.4: Validación física de las cuatro operaciones lógicas.



`SRL 80 01`



`SRA 80 01`

Figura 8.5: Diferencia observada entre los desplazamientos lógico y aritmético.

En los trece ensayos, el patrón de los LEDs coincidió con la salida calculada y con los indicadores previstos en simulación. En particular, el caso `80 + 80` produce `00` y activa simultáneamente Z, C y V. Las fotografías también permiten observar la diferencia entre carry y overflow, así como entre los modos de desplazamiento lógico (SRL) y aritmético (SRA). Además, la carga secuencial mediante pulsadores se comportó de manera estable durante la prueba y no se observaron fallas que exigieran repetir la implementación.

Sección 9

Conclusiones

Por haber sido el primero de la materia, este trabajo práctico representó para el grupo mucho más que la construcción de una ALU funcional. También constituyó el primer contacto con un flujo completo de diseño digital: partimos de una especificación, la expresamos en `SystemVerilog` y recorrimos las etapas necesarias para convertirla en un circuito capaz de ejecutarse sobre una placa. Este recorrido permitió comprender que describir, verificar, sintetizar e implementar no eran nombres distintos para una misma tarea, sino instancias que respondían preguntas diferentes sobre el diseño.

Uno de los aprendizajes más importantes consistió en reconocer que describir hardware no equivale a escribir un programa convencional. El código RTL expresa estructuras que trabajaban de manera concurrente, por lo que decisiones como separar lógica combinacional y secuencial, registrar una entrada o incorporarla al dominio de un reloj tienen una consecuencia física. El contacto con `Vivado` volvió visible esa relación: los esquemas RTL, la síntesis, la utilización de recursos y la implementación muestran distintas representaciones de una misma descripción y ayudan a relacionar el código con el circuito resultante.

La división en módulos y la escritura de bancos de pruebas también modificaron nuestra manera de encarar el problema. Separar la ALU, los sincronizadores y la integración superior no solo ordenó los archivos; estableció límites claros y permitió formular comprobaciones específicas para cada responsabilidad. A su vez, escribir testbenches obligó a definir de antemano qué resultado se esperaba, qué condiciones de borde eran relevantes y cómo detectar un error de forma automática. La verificación dejó así de ser una inspección posterior y pasó a formar parte del propio proceso de diseño.

La carga del bitstream y la utilización de la `Basys 3` completaron ese aprendizaje al trasladar el circuito desde las herramientas de simulación hasta un dispositivo real. Ver que la placa respondió como se esperaba en el primer intento fue una satisfacción para el grupo, pero, sobre todo, confirmó el valor de haber definido con cuidado las interfaces, las restricciones y las pruebas antes de llegar al laboratorio. Los switches, pulsadores y LEDs dejaron de ser señales abstractas y pasaron a mostrar de manera directa las decisiones tomadas en el código.

Como corolario, este trabajo práctico nos dejó una primera metodología para futuros desarrollos digitales. Aprendimos a dividir el problema y a comprender qué hardware describía cada bloque; luego, a verificarlo de manera aislada antes de integrarlo y llevarlo al dispositivo. Más que el resultado particular de una ALU, esa forma de trabajo constituyó el aprendizaje que el grupo consideró más valioso de esta primera experiencia.

Referencias

- Digilent, *Basys 3 Master XDC*. Disponible en `https://github.com/Digilent/digilent-xdc`.
- AMD, documento UG912, *Vivado Design Suite Properties Reference Guide*. Describe la propiedad `ASYNC_REG` y está disponible en `docs.amd.com`.
- Fuentes `SystemVerilog`, testbenches, restricciones y reportes generados por el proyecto del Trabajo Práctico N.º 1.
- Capturas de elaboración, simulación, síntesis e implementación incluidas en este documento.